$\mathbb{A}urum$ Language Standard

@NHProductions on Github

Version AUR26.2

Abstract

This document specifies the syntax, form, and the process of interpretation in the programming language Aurum. Its purpose is to explain how languages work, how that impacts the Aurum programming language, as well the form and style of the Aurum language. While this document isn't a full tutorial, it intends to serve as both a reference for Aurum programmers, as well an intro to the principles of interpreter design.
Sections are included that clarify how various stages of the Aurum interpreter work, the syntax of Aurum, and includes documentation, and usage examples on its standard library functions.
This document will change as the Aurum programming language changes.

Contents

1 Introduction

1. This document can be changed at any point as the language evolves. Clauses will warn implementer if that function could change in the future.

2. This document is divided into several subdivisions: - How the Aurum interpreter works (Sections 2-4) - The Language syntax, contains, and semantics (Section 5) - The Standard library (Section 6)

2 Changed from Last Version

New Features:

- `timeld() -> double128` - Gets current time as a long double. See 8.5
- AurX Pipeline — Added a way to convert bytecode into an executable file, to save time parsing/bytecoding. See 9
- Function pointers — Added a way to turn a function into a literal. See 7.5.2.
- `array_sort()` Modification - Now uses a dedicated sorting function. See 8.1.4

Miscellaneous Changes:

- Changed local variables list into a dynamic array, making 100x100 matrix addition 33% faster. (Forgot to do this in Aurum261.)
- Fixed issue wherein long doubles wouldn't print correctly in build versions.
- Better file management (Organized folders)
- Fixed issue where custom structs containing system structs wouldn't parse correctly.

3 Getting started

3.0.1 Requirements

- Have a windows-based system with admin permissions.
- Atleast 2mb of free storage. (The actual executable is way less than that; this is just an estimation)
- Atleast 1gb of ram (For most programs, it takes about 500-600mb of ram)

3.0.2 Installing Aurum (For users)

1. Aurum only supports windows-based systems.
2. Download Aurum from the github page.
3. Store it in a safe directory (such as C:/Aurum).
4. Add it to your global environment variables. as whatever folder it's in.
5. Restart any terminals you may have open.
6. Now, you're able to run Aurum files by doing `aurum file.aur`. Alternatively, you can make an AurX file by doing `aurum file.aur -x`. This makes a semi-compact semi-executable file that can be used to save interpreted versions of programs. To run an AurX file, do `aurum file.aurx`. For more info, see 9

3.0.3 Building Aurum (For development)

1. Download the github repository.
2. Have VS code, clang, and msys32 installed.
3. Open the x64 console for VS code as administrator permissions.
4. Type in "code" into that console, and it'll open VSCode
5. Open the repository in VSCode.

6. You should now be able to build it. If you want detailed debugging, go to winInclude.h, and enable "isDebug". If you want to build a release version, do "Clang: Aurum build". If you want to debug (and use fsanitizer), use "Clang: Aurum Debug". If isDebug is on, it will automatically open main.aur in the project directory. Make sure to review .vscode/tasks.json & .vscode/launch.json to ensure the clang path is correct.

4 Scope

This document specifies the form and process of interpreting programs written in the Aurum programming language. It is designed to promote the portability of Aurum programs in the Windows ecosystem. It is intended for use by implementers, programmers, and language designers. It specifies:

- How the Aurum interpreter works, including each step of the process.
- The syntax & constraints of the Aurum language.
- The representation of data input & output by Aurum programs.

5 Terms, definitions, and symbols

This section serves as a reference for terms, definitions, and symbols used throughout this document.

1. access (verb): To read or modify the value of an object
2. array (noun): Data structure in which items are contiguous.
3. argument (noun): Value passed into a function.
4. negate (verb): To multiply a number by -1.
5. behavior (noun): External appearance or action.
6. undefined behavior (noun): Behavior that this document imposes no requirements of, or results from the use thereof.
7. bit (noun): Smallest unit of a binary number that represents either 1 or 0.
8. byte (noun): 8 bits.
9. character (noun): A specified amount of bytes that represent a symbol or letter.
10. char (noun): One-byte type that represents an ASCII character (1-255).
11. int (noun): 4 byte numeric type (32 bits)

12. linked list (noun): Data structure in which nodes are not contiguous — they contain pointers to the next node.

13. long (noun): 8 byte numeric type (64 bits)

14. object (noun): Region of memory in execution that can represent values.

15. parameter (noun): Variable declared upon a function's declaration that only gains a value when an argument is passed into the function.

16. short (noun): 2 byte numeric type.

17. signed (adjective): When applied to a type, refers to it being able to support negative numbers.

18. stack (noun): Data structure in which items are either pushed (inserted at index 0), or popped (removed at index 0).

19. unsigned (adjective): When applied to a type, refers to it not being able to support negative numbers.

20. $\lceil x \rceil$ - Arithmetic Ceiling of x (Least integer greater than or equal to x) EXAMPLE: $\lceil 3.055 \rceil = 4$; $\lceil -2.9 \rceil = -2$

21. $\lfloor x \rfloor$ - Arithmetic Floor of x (Greatest integer less than or equal to x) EXAMPLE: $\lfloor 3.055 \rfloor = 4$; $\lfloor -2.9 \rfloor = -3$

22. x**y - x raised to the power of y x^y

23. x//y - x floor-divided by y $\lfloor \frac{x}{y} \rfloor$

24. $x\%y$ - x modulo y. (remainder of x/y)

6 Aurum Interpreter

This section is about how the Aurum Interpreter transforms plain text code and executes it. It's recommended that you at least have a surface-level knowledge of Aurum's syntax before reading this section.

6.1 Lexing

The first stage of processing is called lexing (Also called tokenizing). During this stage, the code gets turned into an array of tokens. They have a type associated with them, and a value.

```
typedef enum {
 T_NONE=0,
 T_IDENTIFIER,
 T_OPERATOR,
 T_STRING,
 T_CHAR,
 T_NUMBER,
 T_KEYWORD,
 T_DELIMITER,
 T_GROUPING,
 T_LIBRARY
} tokenType;
typedef struct token {
 tokenType type;
 char* value;
} token;
```

Figure 1: Definitions of the token struct & tokenType enum

Essentially, the code gets turned into a list of number-string pairs that represent a token's type & text.

6.1.1 Example

The example sections in this will use the fizzbuzz problem. If you're unfamiliar with it, it's a test used for new-programmers, as well as programming languages. The task is to make a program that does the following:

- Count sequentially, starting from 1.
- If a number is divisible by three, print "fizz".
- If a number is divisible by five, print "buzz".
- If a number is divisible by both three and five, print "fizzbuzz".
- Otherwise, print that number.

Example:

```
1, 2, fizz, 4, buzz, 5, fizz, 7, 8, 9, buzz, 10, 11, fizz, 13, 14, fizzbuzz
```

```
function fizzbuzz(int amt) -> void {
 for (int n = 1; n <= amt; n++) {
  if ( (n % 3 == 0) && (n % 5 != 0) ) {
   print("fizz\n");
  }
  elif ( (n % 5 == 0) && (n % 3 != 0) ) {
   print("buzz\n");
  }
  elif ( (n % 5 == 0) && (n % 3 == 0) ) {
   print("fizzbuzz\n");
  }
  else {
   print("%i\n", n);
  }
 }
}
```

Figure 2: Fizzbuzz code

The lexer then lexes Figure 2, which is shown below. It's in the format of TYPE(text).

```
   [KW(function), IDENTIFIER(fizzbuzz), GROUPING((),
KW(int), IDENTIFIER(amt), GROUPING()) KW($\rightarrow$)
KW(void) GROUPING({), KW(for), GROUPING((), KW(int),
IDENTIFIER(n), OP(=), NUMBER(1), DELIM(;), IDENTIFIER(n),
OP(<=), IDENTIFIER(amt), DELIM(;), IDENTIFIER(n), OP(++),
GROUPING()), GROUPING({), KW(if), GROUPING((), GROUPING((),
IDENTIFIER(n), OP(%), NUMBER(3), OP(==), NUMBER(0),
GROUPING()), OP(&&), GROUPING((), IDENTIFIER(n), OP(%),
NUMBER(5), OP(!=) NUMBER(0), GROUPING()), GROUPING()),
GROUPING({), IDENTIFIER(print), GROUPING((), STRING(fizz\n),
GROUPING()), DELIM(;), GROUPING(}),KW(elif), GROUPING((),
GROUPING((), IDENTIFIER(n), OP(%), NUMBER(5), OP(==),
NUMBER(0), GROUPING()), OP(&&), GROUPING((), IDENTIFIER(n),
OP(%), NUMBER(3), OP(!=), NUMBER(0), GROUPING()), GROUPING()),
GROUPING({), IDENTIFIER(print), GROUPING((), STRING(buzz\n),
GROUPING()), DELIM(;), GROUPING(}), KW(elif), GROUPING((),
GROUPING((), IDENTIFIER(n), OP(%), NUMBER(5), OP(==),
NUMBER(0), GROUPING()), OP(&&), GROUPING((), IDENTIFIER(n),
OP(%), NUMBER(3), OP(==), NUMBER(0), GROUPING()),
GROUPING()), GROUPING({), IDENTIFIER(print), GROUPING((),
STRING(fizzbuzz\n), GROUPING()), DELIM(;), GROUPING(}),
KW(else), GROUPING({), IDENTIFIER(print), GROUPING((),
STRING(%i), DELIM(,), IDENTIFIER(n), GROUPING()), DELIM(;),
GROUPING(}), GROUPING(}), GROUPING(})]
```

Figure 3: Lexed fizzbuzz code.

If you want to know more details about the lexing process, look at lexer.c in Aurum's source code, or the wikipedia page on lexical analysis.

6.2 Parsing

The next stage of processing is called parsing. During this stage, the list of tokens gets turned into an Abstract Syntax Tree. An AST is a tree that represents a program. Nodes are usually composed of a type, value, and a list of child nodes. Here's what Aurum uses for ASTNodes:

```
typedef union {
 double numberVal;
 num numVal;
 char* nameVal;
 char* opVal;
 char* anyVal;
} ASTValue;
typedef struct ASTNode {
 ASTValue value;
 struct ASTNode* left;
 struct ASTNode* right;
 List* children;
 ASTType type : 8; // Types not shown, but ASTType is an enum.
} ASTNode;
```

There's several important consequences of parsing, which make later stages much easier.

- Delimiters are removed in this stage
- Grouping symbols are removed.
- Parsing happens in a way that makes it so that doing PEMDAS isn't necessary (the tree already has PEMDAS implemented)
- +=, -=, *=, /=, and other operator-assignment are turned into "= +"; e.x a += b turns into a = a + b

From highest precedence to least precedence, Aurum uses: $Unary > Exponentiation > Multiplication/Division > Addition/Subtraction > Bitshift > Comparison > Equality > BitwiseOperators > LogicalOperators > equality$

6.2.1 Example

The entire fizzbuzz function is a little bit too big to make into a tree and show here, so here's the parsing of the 1st if statement.

```
Converted Below: if ( (n % 3 == 0) && (n % 5 != 0))
```

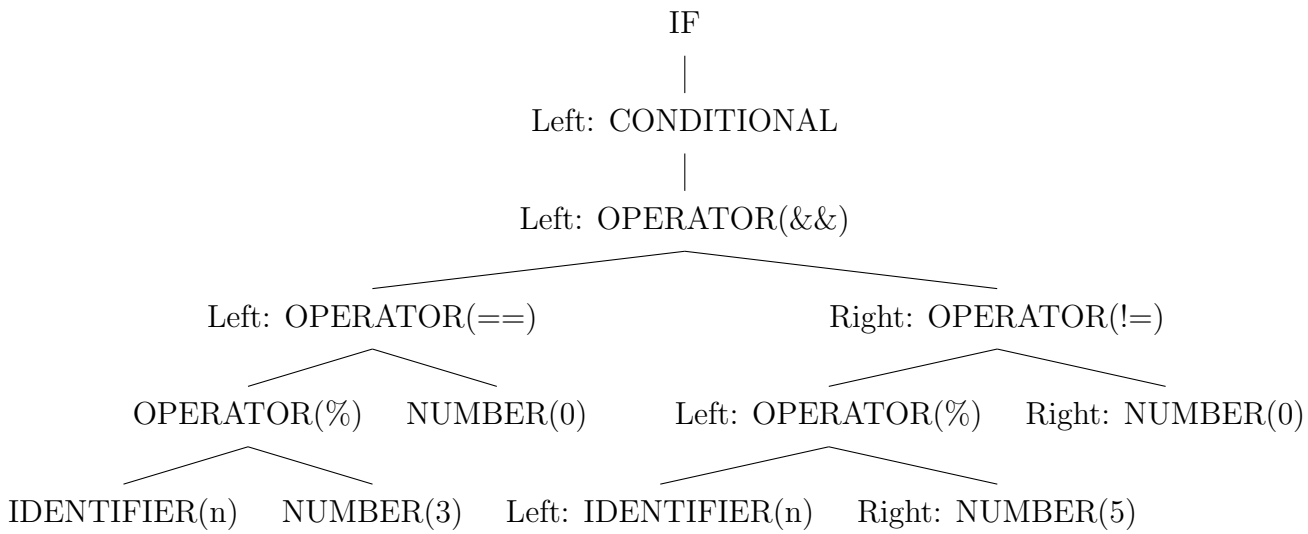


Figure 4: 1st if statement in fizzbuzz() parsed

If you want to know more details about the parsing process, look at parser.c in Aurum's source code, or the wikipedia page on Abstract Syntax Trees.

6.3 Bytecoding & Execution

While an AST could be executed just from the tree (Like it is in Perl & Ruby), it's generally better to convert it to another form — Bytecode. Named for each instruction only being a byte, bytecode is much more efficient than just traversing the AST. Below are the structs used for storing bytecode.

```
  typedef struct {
   List* instructions;
   char* name;
  } chunk;
  typedef enum opCode {. . . implementation not shown; see Stack
Instructions}
  typedef struct {
   int* args;
   char argc;
   char code;
  } instruction;
  typedef struct {
   List* globals;
   List* constants;
   List* functionIdentifiers;
   List* chunks;
   List* structDefs;
  } byteCode;
```

Notice how each instruction is only 10 bytes (padded to 16 bytes), which is a large improvement compared to an ASTNode being 64 bytes. Each instruction is meant to be simple almost-assembly-like instruction acting on a stack of values. Functions are parsed as a separate chunk, then inserted into bytecode.chunks. Struct attributes are parsed into a struct definition, and their functions are parsed as global functions with names in the format of structName_functionName.
In addition, floor division & exponentiation are converted into calls to !op-Func (see standard library).

6.3.1 Stack instructions

Below is the full list of stack instructions used by the bytecoder. In this list, letters are used to symbolize the position of a value before being popped from the stack. x is a numeric parameter for the instruction.

- LOAD CONST x — Push constants[x] onto the stack
- LOAD GLOBAL x — Push a globals[x] onto the stack.
- STORE GLOBAL x — Pop the stack → a; store a into globals[x].
- LOAD LOCAL x — Push a locals[x] onto the stack.
- LOAD IDX — Pop the stack twice → a & b. Push b[a] onto the stack.
- STORE IDX — Pop the stack thrice → a, b, c. Store a into c[b].
- ARRAY LEN — Pop the stack → a. Push the length of a onto the stack. This is the # operator.
- BUILD ARRAY x — Pop the stack x times. Using the values popped, build an array. Push that array onto the stack.
- NEW STRUCT x — Push a uninitialized struct with structDefinitions[x] onto the stack.
- GET FIELD x — Pop the stack → a. Push a.x onto the stack.
- ADD — Pop the stack twice → a & b. Push $a + b$ onto the stack.
- SUB — Pop the stack twice → a & b. Push $a - b$ onto the stack.
- MUL — Pop the stack twice → a & b. Push $a * b$ onto the stack.
- DIV — Pop the stack twice → a & b. Push a/b onto the stack.
- NEG — Pop the stack once → a. Push $-a$ onto the stack.
- MOD — Pop the stack twice → a & b. Push a % b onto the stack.
- BXOR — Pop the stack twice → a & b. Push a $\hat{}$ b onto the stack.
- BAND — Pop the stack twice → a & b. Push a & b onto the stack.

- BNOT — Pop the stack once → a. Push a onto the stack.
- BOR — Pop the stack twice → a & b. Push a | b onto the stack.
- LSHIFT — Pop the stack twice → a & b. Push a << b onto the stack.
- RSHIFT — Pop the stack twice → a & b. Push a >> b onto the stack.
- GTHAN — Pop the stack twice → a & b. Push $a > b$ onto the stack.
- LTHAN — Pop the stack twice → a & b. Push $a < b$ onto the stack.
- LEQTHAN — Pop the stack twice → a & b. Push $a \geq b$ onto the stack.
- GEQTHAN — Pop the stack twice → a & b. Push $a \leq b$ onto the stack.
- EQUALS — Pop the stack twice → a & b. Push a == b onto the stack.
- OR — Pop the stack twice → a & b. Push a||b onto the stack.
- AND — Pop the stack twice → a & b. Push a && b onto the stack.
- NOT — Pop the stack once → a. Push !a onto the stack.
- XOR — Pop the stack twice → a & b. Push a xor b onto the stack.
- CALL x y - Pop the stack y times. Call x, passing those arguments into x as a of parameters
- JUMP x — Jump to line x.
- JUMP IF FALSE x — Pop the stack once → a. If a == 0, jump to line x.
- HALT — Exits the program
- FPTR x — Makes an fnptr struct that points to x.
- DCALL x — Pops the stack once → a. Pop the stack x times. Call a with x arguments.

6.3.2 Example

Going back to the original fizzbuzz program, below is the bytecode for it. Indents & comments have been added for increased readability.

```
for loop
Initialization: i=1
0. LOAD CONST 1 - Load 1 onto the stack
1. STORE LOCAL 1 - Store 1 into local 1 (i)
2. JUMP 11 - Jump to line 11 (1st time the for loop happens)
Update i: i++
3. LOAD LOCAL 1 - Load i onto the stack
4. LOAD CONST 37 - Load 1 onto the stack
5. ADD - Remove i & 1 from the stack. add i+1 to the stack.
6. STORE LOCAL 1 - Store i+1 into i.
Check condition: i <= amt
7. LOAD LOCAL 1 - Load i onto the stack
8. LOAD LOCAL 0 - Load amt onto the stack
9. LEQTHAN - Remove i and amt from the stack. add (i <= amt) onto the stack.
10. JUMP IF FALSE 63 - Remove (i <= amt) from the stack. If that equals zero, jump to line 63 (ending the loop).
1st if condition (i % 3 == 0) && (i % 5 != 0)
1st part of the and statement (i % 3 == 0)
11. LOAD LOCAL 1 - Load i onto the stack
12. LOAD CONST 38 - Load 3 onto the stack
13. MOD - Remove i & 3 from the stack, and put i % 3 onto the stack
14. LOAD CONST 0 - Load 0
15. EQUALS - Remove 0 & (i % 3), and add (i % 3 == 0) onto the stack.
2nd part of the and statement (i % 5 != 0)
16. LOAD LOCAL 1 - Load i onto the stack
17. LOAD CONST 39 - Load 5 onto the stack
18. MOD - Remove i & 5 from the stack, and put i % 5 onto the stack.
19. LOAD CONST 0 - Load zero onto the stack
20. EQUALS - Remove (i % 5) & zero from the stack. Add (i % 5 == 0) to the stack.
21. NOT - Remove (i % 5 == 0) from the stack. Add !(i % 5 == 0)
22. AND - Remove !(i % 5 == 0) and (i % 3 == 0) from the stack. Add (!(i % 5 == 0) && (i % 3 == 0))
23. JUMP IF FALSE 27 - Remove (!(i % 5 == 0) && (i % 3 == 0)) from the stack. If that value is 0, jump to 27.
1st if block (print "fizz")
24. LOAD CONST 40 - Add the string "fizz" to the stack.
25. CALL 7 1 - Remove 1 thing from the stack, and call function 7 (print)
26. JUMP 62 - Jump to line 62.
2nd if condition (i % 5 == 0) && (i % 3 != 0)
Load 1st part of the and statement (i % 5 == 0)
27. LOAD LOCAL 1 - Load i onto the stack
28. LOAD CONST 39 - Load 5 onto the stack
29. MOD - Remove i & 5 from the stack. Add (i % 5) onto the stack.
30. LOAD CONST 0 - Load zero onto the stack.
31. EQUALS - Remove (i % 5) and 0 from the stack. add (i % 5 == 0) to the stack.
Load 2nd part of the and statement (i % 3 != 0)
32. LOAD LOCAL 1 - Load i onto the stack.
33. LOAD CONST 38 - Load 3 onto the stack.
34. MOD - Remove i & 3 from the stack. add (i % 3) to the stack
35. LOAD CONST 0 - Load zero onto the stack.
36. EQUALS - Remove (i % 3) & zero from the stack. add (i % 3 == 0) to the stack.
37. NOT - Remove (i % 3 == 0) from the stack. Add !(i % 3 == 0)
38. AND - Remove !(i % 3 == 0) and (i % 5 == 0) from the stack. Add (!(i % 3 == 0) && (i % 5 == 0)) to the stack.
39. JUMP IF FALSE 43 - Remove (!(i % 3 == 0) && (i % 5 == 0)) from the stack. If that equals zero, jump to line 43.
2nd if block (print "buzz"):
40. LOAD CONST 41 - Load "buzz" onto the stack.
41. CALL 7 1 - Remove 1 parameter from the stack and call function 7 (print)
42. JUMP 62 - Jump to line 62.
3rd if condition (i % 5 == 0 && i % 3 == 0):
Load 1st part of the and statement (i % 5 == 0)
43. LOAD LOCAL 1 - Load i onto the stack
44. LOAD CONST 39 - Load 5 onto the stack
45. MOD - Remove i & 5 from the stack. Add (i % 5) to the stack.
46. LOAD CONST 0 - Load zero onto the stack.
47. EQUALS - Remove (i % 5) & zero from the stack. add (i % 5 == 0)
Load 2nd part of the and statement (i % 3 == 0)
48. LOAD LOCAL 1 - Load i onto the stack.
49. LOAD CONST 38 - Load 3 onto the stack.
50. MOD - Remove i & 3 from the stack. add (i % 3) to the stack.
51. LOAD CONST 0 - Load zero onto the stack.
52. EQUALS - Remove (i % 3) and zero from the stack. Add (i % 3 == 0) to the stack.
53. AND - Remove (i % 3 == 0) and (i % 5 == 0) from the stack. add (i % 3 == 0 && i % 5 == 0) to the stack
54. JUMP IF FALSE 58 - Remove (i % 3 == 0 && i % 5 == 0) from the stack. Jump to line 58 if it's equal to zero.
3rd if block (print "fizzbuzz"):
55. LOAD CONST 42 - Add "fizzbuzz" to the stack.
56. CALL 7 1 - Remove "fizzbuzz" from the stack, and call function 7 (print).
57. JUMP 62 - Jump to line 62.
else block (print the number):
58. LOAD CONST 43 - Load "%i" onto the stack.
59. LOAD LOCAL 1 - Load i onto the stack
60. CALL 7 2 - Take two parameters from the stack (%i and i), and call print().
61. JUMP 62 - Jump to line 62
start the loop again:
62. JUMP 3 - Jump to line 3
```

7 Language Standard

7.1 Types

The meaning of a value is determined by its type. Below is a list of types in Aurum. The following types are typed to store integer numbers.

1. bool — Stores either true or false.
2. char/int8 — Stores integers in the range [-127, 127]. Has 1 byte. Is used to also map numbers to ASCII characters.
3. uchar/uint8 — Stores integers in the range [0, 255]. Has 1 byte. Is used to also map numbers to ASCII characters.
4. short/int16 — Stores integers in the range $[-2^{15} - 1, 2^{15} - 1]$. Has 2 bytes.
5. ushort/uint16 — Stores integers in the range $[0, 2^{16} - 1]$. Has 2 bytes.
6. int/int32 — Stores integers in the range $[-2^{31} - 1, 2^{31} - 1]$. Has 4 bytes.
7. uint/uint32 — Stores integers in the range $[0, 2^{31} - 1]$. Has 4 bytes.
8. long/int64 — Stores integers in the range $[-2^{63} - 1, 2^{63} - 1]$. Has 8 bytes.
9. ulong/uint64 — Stores integers in the range $[0, 2^{63} - 1]$. Has 8 bytes.

The following types are used to store decimal numbers. They have a set amount of bits for exponent, and a set amount of bits for their mantissa, and 1 bit for signage.

1. float/double32 — Stores decimal numbers. Has 23 bits for its mantissa, 8 bits for its exponent, and 1 bit for signage.
2. double/double64 — Stores decimal numbers. Has 47 bits for its mantissa, 16 bits for its exponent, and 1 bit for signage.
3. longdouble/double128 — Stores decimal numbers. Has 95 bits for its mantissa, 32 bits for its exponent, and 1 bit for signage.

The user can also define custom types (structs). For more info on that, see the section on structs. In addition, there's serveral system-defined structs. Below are those imported in the default library:

1. string - Used to store arrays of characters. Has to end with a null terminator (0).
2. exception - Used to store runtime errors.

7.2 Lexical Elements

A Lexical Element (or a token) is the smallest unit that is read by the lexer. Below is the various types of lexical units, and what happens to them as the interpretation process takes place.

7.2.1 Keywords

A keyword is a reserved identifier for Aurum. There's 47 keywords in Aurum, listed as follows:

1–26. Types used by the user to declare variables.

27. NaN — Floating-point constant that signifies "Not a Number".
28. inf — Floating-point constant used to represent infinities.

29 & 30. Boolean true & false

31–33. If, else, and elif — Used to declare conditional statements.

34 & 35. Switch & Case — Used to declare switch/case statements.

36–39. For, While, break, & continue — Used to declare loops.

40—42 Function, return, `->`, & void — Used to declare functions.

43. struct — Used to declare structs.
44. import — Used to import other files & libraries.
45. const — Used to make variables immutable.
46. auto — Used to make it so that the type for the variable is resolved at runtime.

7.2.2 Number Literals

Number literals can be defined by having a type suffix after them:

```
  12345c - Coerces a number to a char.
  12345uc - Coerces a number into a uchar.
  12345s - Coerces a number to a short.
  12345us - Coerces a number to a ushort
  12345i - Coerces a number to an int.
  12345ui - Coerces a number to a uint.
  12345l - Coerces a number to a long.
  12345ul - Coerces a number to a unsigned long
  12345.02f - Coerces the number to a float.
  23256.5d - Coerces the number to a double64.
  5325.999ld - Coerces the number to a double128.
```

Integers can also be defined by having a base prefix before the number. This is undefined behavior for decimal types:

```
  0xFFA12 - Interprets the number as a hex literal (Converts it into
a decimal number during tokenization)
  0d91247 - Interprets the number as a decimal literal (Does nothing)
  0o77127 - Interprets the number as a octal literal (Converts it
into a decimal number during tokenization)
  0b10110 - Interprets the number as a binary literal (Converts it
into a decimal number during tokenization.
```

All stages of the interpretation process have some method to store numbers. Most of these stages employ the num struct (see Figure 5) to store numbers regardless of type.

```
typedef struct {
 numberTypes type;
 union {
  char cVal; // 1 byte
  unsigned char ucVal; // 1 byte
  short sVal; // 2 bytes
  unsigned short usVal; // 2 bytes
  int iVal; // 3 bytes
  unsigned int uiVal; // 3 bytes
  int64_t lVal; // 8 bytes (64 bits)
  uint64_t ulVal; // 8 bytes (64 bits)
  float fVal; // 4 bytes (32 bits)
  double dVal; // 8 bytes (64 bits)
  long double ldVal; // 16 bytes (128 bits)
  bool bVal; // 1 byte
 } value;
} num;
```

Figure 5: num struct

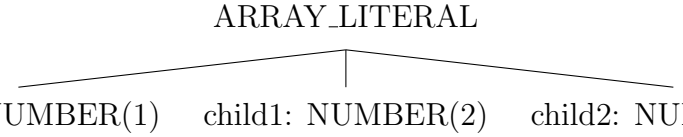
7.2.3 Array Literals

Array literals can be declared in the following format: `[elementOne, elementTwo, ...]`. Literals have to be surrounded by brackets, and have commas seperating elements. In addition, array literals are only allowed to have one unique type (e.x, you can't have an array literal with both strings and integers).

During lexing, array literals are tokenized as:

`[1, 2, 3] -> GROUPING([) NUMBER(1) COMMA(,) NUMBER(2) COMMA(,) NUMBER(3), GROUPING(]).`

Which then, during parsing, are turned into:



During bytecoding, if they are able to be turned into a constant, they are. If not, they are build during execution using BUILDARRAY instruction.

7.2.4 String & Character Literals

Strings & Character literals are declared by using quotes. Double-quotes signify a string, and single-quotes signify a character. Single-quote strings are only allowed to have one character. Strings can only have ASCII characters (wide-strings have not been implemented yet. See section 9.0.2).

Several escape sequences have been defined for characters that are inconvenient for strings. Below are the full list of escape sequences:

Escape sequence	Character produced	Locale
`\n`	newline	Any
`\t`	tab	Any
`\"`	”	Any
`\'`	’	Any
`\\`	\	Any
`\e`	€ (Euro)	English-US 1252
`\tm`	™	English-US 1252
`\em`	— (EM dash)	English-US 1252
`\en`	– (EN dash)	English-US 1252
`\c`	¢	English-US 1252
`\p`	£	English-US 1252
`\y`	¥	English-US 1252
`\s`	§	English-US 1252
`\cp`	©	English-US 1252
`\r`	®	English-US 1252
`\pm`	±	English-US 1252
`\mi`	μ	English-US 1252
`\pa`	¶	English-US 1252
`\x00`	Grab an ASCII char using hex code.	Any

Figure 6: Table of escape sequences

7.3 Comments

A line can be commented out using ‘ (grave). example:

```
int a = 2; ‘ a = 5;
print("%i", a); ‘ prints "2".
```

7.4 Operators

An operator is a symbol used to modify data.

7.4.1 Binary Arithmetic Operators

The binary arithmetic operators are as follows:

- +; computes $a + b$ e.x $2 + 3 = 5$
- Binary -; computes $a - b$ e.x $5 - 2 = 3$
- Unary -; computes $-a$ e.x $-2 = -2$
- *; computes $a * b$ e.x $5 * 3 = 15$
- /; computes a/b e.x $6/2 = 3$
- %; computes $a\%b$ e.x $7\%3 = 1$

The following are cases that you might want to be aware of if you are dealing with binary arithmetic operators.

1. If at any point the program attempts to do n/0, it will throw an error.
2. If a & b are unsigned integers, and $b > a$, doing $a - b$ will NOT throw an error. It will silently wrap around. (e.x $3ui - 5ui = 253ui$).
3. If a or b aren't integers, $a\%b$ will throw an error.

7.4.2 Bitwise Operators

A bitwise operator is an operator that modifies a number's binary data (or bits). They are as follows:

- |; computes bitwise or (A bit is 1 in the result if that position is 1 in one or both of the numbers). e.x $0b101101|0b110000 = 0b111101$
- &; computes bitwise and (A bit is 1 in the result if that position is 1 in both of the numbers). e.x $0b101101\&0b110000 = 0b100000$
- ^; computes bitwise xor (A bit is 1 in the result if that position is 1 in exactly 1 of the numbers). e.x $0b101101\hat{}0b110000 = 0b011101$
- ~; computes bitwise not (A bit is 1 in the result if that bit is 0 in the number). e.x ~0b101101 = 0b010010.
- << - Computes right bitshift (adds n zeroes). e.x 0b11 << 2 = 0b1100.
- >> - Computes left bitshift (removes n digits) e.x 0b1101 >> 2 = 0b11.

The following are some cases you might want to be aware of if you are dealing with binary arithmetic operators.

1. All of these only work with integers.
2. Shifting negative numbers is undefined behavior.

7.4.3 Logical Operators

A logical operator is an operator whose parameters are booleans or whose result is a boolean. They are as follows:

- ||; computes logical or. a || b is true if at least one input is true.
- &&; computes logical and. a && b is true if a and b are true.
- ^^; computes logical xor. a ^^ b is true if either a or b are true, but not both.
- !!; computes logical not. !a is true if a is false.
- >>; computes greater than. a > b is true if a is greater than b.

- `<<`; computes less than. a `<` b is true if a is less than b.
- `>=` / `=>`; computes greater than. a `>=` b is true if a is greater than or equal to b.
- `<=` / `=<`; computes less than. a `<=` b is true if a is less than or equal to b.
- ==; computes equality. a == b is true if a equals b.
- !=; computes inequality. a != b is true if a isn't equal to b.

7.4.4 Other Operators

These are operators who have special conditions or don't really fit with other categories.

- **; computes exponentiation; returns a^b. Operations that result in a complex number throw an error (see section 8.3)
- //; computes floor division; returns $\lfloor \frac{a}{b} \rfloor$.
- #; computes array length. e.x #[2,4,6,8] = 4.

7.4.5 Assignment Operators

The assignment operator is =. It assigns the value of the right side to the left. Any operator except unary operators & logical operators can be turned into an assignment operator by appending an =. e.x + → +=.

7.4.6 Operators during interpretation

During lexing, operators are processed as seperate tokens. In parsing, assignment operators are broken up into constituent parts (e.x a += b → a = a+b). During bytecoding, // and ** are turned into !opFunc calls (as they are a bit too complicated to just turn into a opcode.)

7.5 Declarations

7.5.1 Variable Declarations & Assignment

A variable can be declared in the following format: `specifiers? type name = value;`. That variable is then assigned to its local scope. Variables in other scopes can have the same name as the variable, and are unable to access the original variable. Having variable specifiers is optional, especially as there's only one right now — const. The const specifier makes a variable immutable, throwing an error if it's changed.

After a variable has been declared, it can be changed if the current scope has access to it. That can be done by doing this format: `name = value;`.

Variables can be referred to anywhere in their scope.

During lexing, variables are treated as `[KW(specifier), KW(type), IDENTIFIER(name), OPERATOR(=) NUMBER(value)]`.

In parsing, declarations are parsed as the following: Then, during byte-

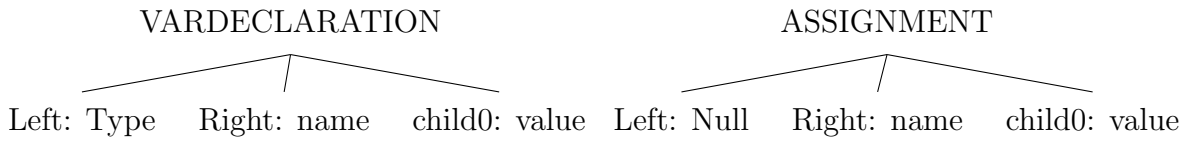


Figure 7: Variable declarations during parsing

coding, assignment & declaration are converted into one instruction: STORE_LOCAL & STORE_GLOBAL (depending on the scope of the variable).

7.5.2 Function Declarations

Functions are declared in the following format:

```
function name(typeParamA paramAName, . . .) -> returnType {
 function block stuff
}
```

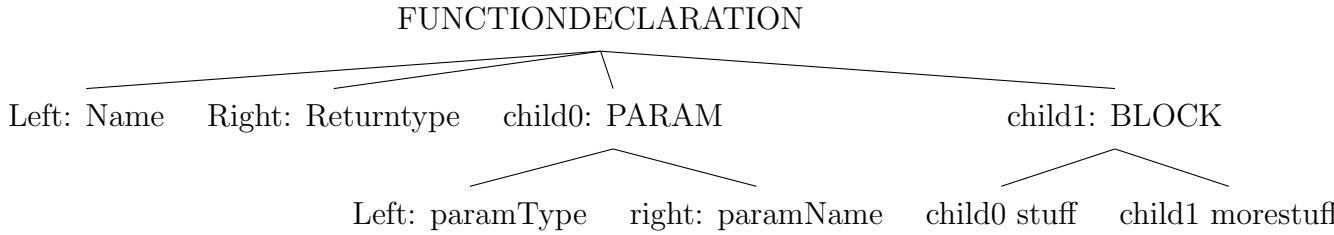
The name can be any identifier, same with parameter names.
returnType can either be a normal type, or void (which means it doesn't return anything). Functions do create their own scope, and they can be called from anywhere. Functions cannot modify their own variables locally Recursion is allowed. Infinite recursion is allowed.

```
function n(int a) -> void {
 a = 5; ‘ ILLEGAL
}
```

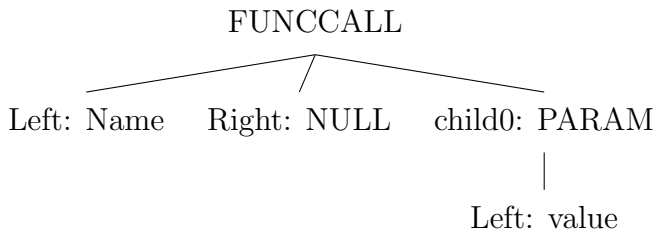
Functions can be called from anywhere in the program.

In addition, a function called main() has to be declared, as it's the entry-point for execution.

During parsing, function declarations are parsed into this format:



Function calls are parsed into this format:



Then, during bytecoding, instructions inside of function declarations are processed into chunks of code added to the bytecode struct. Function calls are processed into `CALL x y`, where x is the function index, and y is the amount of arguments passed.

It's also possible to turn functions into variables. Doing `&functionName` makes a function pointer (fnptr), which then can be used to dynamically call functions. A function pointer can be called by doing `ptrName()`.. If a fnptr called returns a struct, it's illegal to access the members & functions of that struct, without first assigning it to a variable.

Example:

```
Example 1:
function add(int a, int b) -> int {
 return a+b;
}
fnptr a = &add;
a(2,3);
Example 2:
function str() -> string {
 return "Hello World!";
}
fnptr b = &str;
b().toUpper(3) ‘ ILLEGAL
string c = b();
c.toUpper(3) ‘ Legal
```

During parsing, & is treated as a unary operator. Then, during bytecoding, it gets processed into `FPTR x`, where x is the function's index. Calls to dynamic functions get processed into `DCALL y`, where y is the amount of arguments. Below is example 1 converted into bytecode.

```
add (idx 0):
 1. LOAD LOCAL 0 (a)
 2. LOAD LOCAL 1 (b)
 3. ADD
 4. RETURN
main (idx 1):
 1. FPTR 0
 2. STORE LOCAL 0
 3. LOAD CONST 0 (2)
 4. LOAD CONST 1 (3);
 5. LOAD LOCAL 0 (&add)
 6. DCALL 2
```

7.5.3 Struct Declarations

A struct is a datatype that contains other datatypes (members), and functions. They can be used to represent more complex structures. Below is how structs are declared in Aurum:

```
struct name {
 int attribute1; ‘ Initializing members isn’t allowed.
 int attribute2;
 string attribute3;
 function name(int attr1, int attr2, string attr3) -> name {
  this.attribute1 = attr1;
  this.attribute2 = attr2;
  this.attribute3 = attr3;
  return this; ‘ Important: Constructors must always return this.
 }
 function anotherFunc() -> int {
  return this.attribute1*this.attribute2;
 }
}
```

- To access a member of a struct, do `structName.attributeName`.
- To call a function from a struct, do `structName.functionName()`.
- To declare a variable with type struct, do `structName varName = structName()`.
- All struct functions have a hidden parameter — this. Anytime `sp.functionName(param1, param2)` is called, three arguments are actually passed; `structName_functionName(sp, param1, param2)`, and the current struct can be referenced by using `this`
- A function whose name is the same as a struct is called a constructor. e.x `structName var = structName()`. Constructors are used to initialize a struct’s values.

– There are a total of 25 predefined function names that are reserved for operator overloads. They are:

Operator	Function name
+	op_add
Binary minus	op_sub
Unary minus	op_neg
*	op_mul
/	op_div
&&	op_and
||	op_or
^^	op_xor
!=	op_neq
==	op_eq
>=	op_geqthan
<=	op_leqthan
>	op_gthan
<	op_lthan
Unary !	op_not
&	op_band
|	op_bor
^^	op_bxor
Unary ~	op_bnot
<<	op_lshift
>>	op_rshift
%	op_mod
//	op_fdiv
**	op_exp
Unary #	op_len

Example usage:

```
struct a {
 int numa;
 int numb;
 ‘ Constructor not shown
 function op_add(a n) -> a {
  return a(n.numa + this.numa, n.numb + this.numb);
 }
 function b() -> int {
  return 2;
 }
}
...
a(1,2) + a(3,2) == a(4,4)
(a(1,2) + a(3,2)).b ‘ ILLEGAL, operator overloading only
works with simple expressions (No func calls, struct accesses,
or array accesses.). The alternative here would be a(3,2).op_add(a(1,2)).b
```

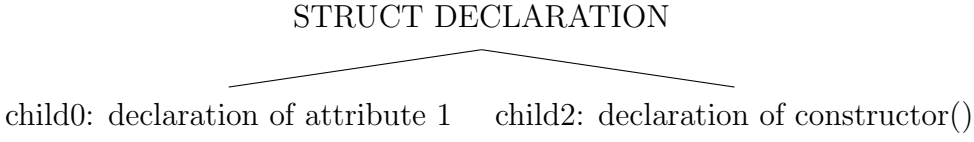


Figure 8: How struct declarations are parsed

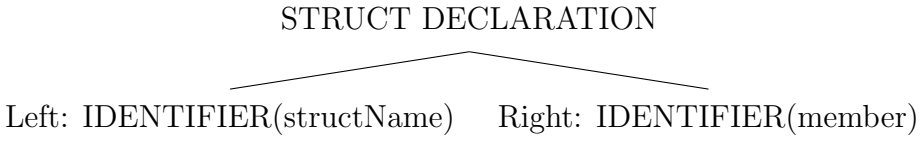


Figure 9: How struct accessing is parsed

– Struct functions are parsed differently from other functions: the function's name is changed to `structName_functionName`, and they gain an extra parameter. This means that one could add custom functions to system structs, simply by declaring a function named `array_functionName` or `string_functionName`. Struct functions are turned into `CALL x y` during bytecoding.

– A struct's variables & name are moved into a structDefinition, which is used to see if a struct access is valid or not. Struct accessing is turned into `GET FIELD x` or `SET FIELD x`.

– During bytecoding, Strings and arrays becoming their own structs, which contain standard-library functions.

7.5.4 Array Assignment

- To declare an array, do `type[] name = []`. This also applies to multidimensional arrays; `double128[] multidim = [[2ld, 3ld, 4ld], [5ld, 6ld, 7ld]]`.
- Arrays can be accessed or assigned to by indexing them `array[n]`, where n is the position within the array that is accessed, starting from 0. This also applies to multidimensional arrays, where `array[n][m] = value` is valid, as well as `array[n] = array2`.

During parsing, array accessing is turned into

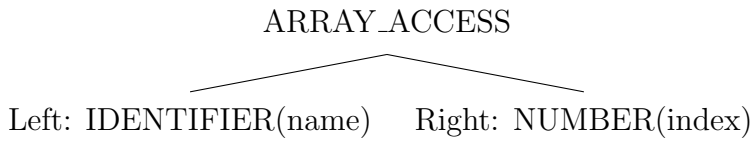


Figure 10: How array access is parsed

During bytecoding, array access / assignment are turned into `LOAD IDX x` and `STORE IDX x`.

7.6 Statements

A statement is a keyword or combination of keywords that are used to implement logic into a program.

7.6.1 Selection Statements

A selection statement is a statement that executes different conditions based on the values of different booleans. In Aurum, there are two types of selection statements: if-statements, and switch-case statements.

An if statement executes a block of code based on the value of a condition.

```
if (condition) {
 ‘ block
}
```

In the above example, condition has to be any boolean value or expression that results in a boolean value. In addition to if statements, elif and else blocks can also be attached to it.

```
if (condition) {
 block
}
elif (anotherCondition) {
 anotherblock
}
elif (athirdcondition) {
 athirdblock
}
else {
 elseBlock
}
```

The important thing about if-else statements is that ONLY one of the blocks will be executed. If statements can have an infinite amount of elif & blocks. There's only allowed to be one if block, and one else-block in an if-else statement.

During parsing, if statements are parsed as the following:

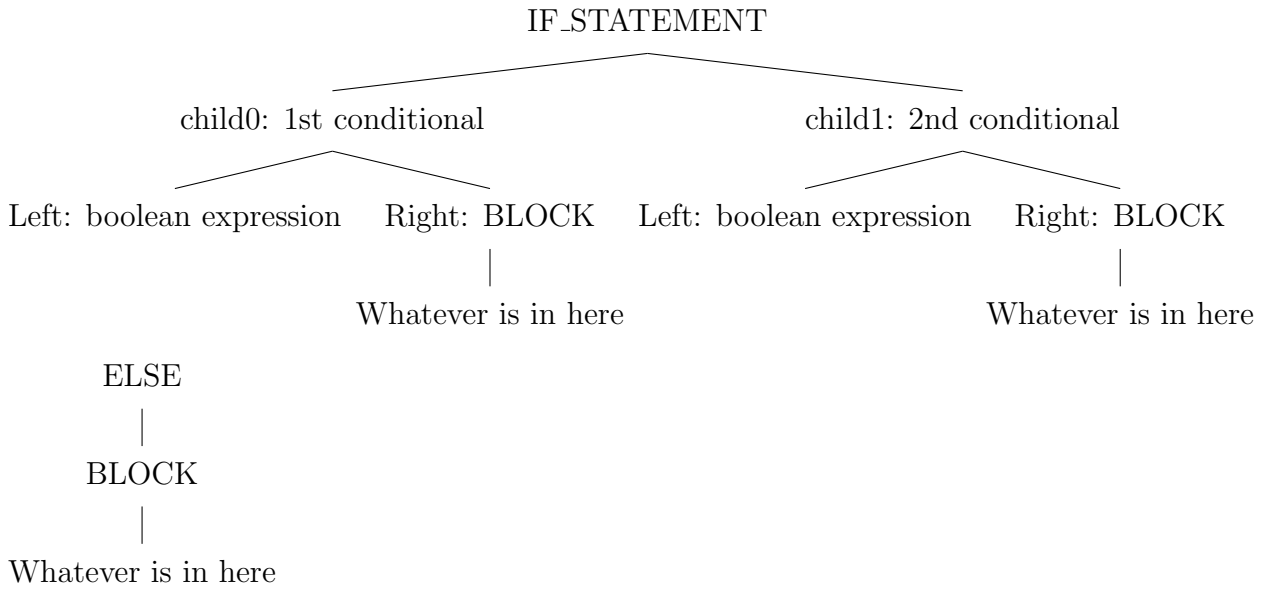


Figure 11: How if, elif, & else statements are parsed.

During bytecoding, they are parsed as a collection of jump statements (see below)

```
If statement:
1. Not shown: Load condition 1
2. JUMP_IF_FALSE 5
3. ; Not shown: Executes If block 1
4. JUMP 10
Elif statement
5. ; Not shown: Load condition 2
6. JUMP_IF_FALSE 9
7. ; Not shown: Executes If block 2
8. JUMP 10
Else statement:
9. ; Not shown; Else block
After the if statement:
10. HALT
```

There's also another type of selection statement: switch statements. They are essentially a simpler if-else statement, and they get parsed similarly. Benefits of using switch statements include increased readability & having simpler code. Below is a switch statement in Aurum.

```
switch (value) {
 case (condition) {
  block1
 }
 case (value) {
  block2
 }
}
```

Switch statements can have an infinite amount of blocks. Each case statement can have ANY boolean expression, or just a literal. Below is an example of a switch statement:

```
int a = 20;
switch (a) {
 case (19) {  ‘ Values can be used
  block1;
 }
 case (a == 10)  { ‘ Any boolean expression can be used
  block2;
 }
 case (a % 2 == 0) {
  block3;
 }
}
```

Default statements have not been implemented yet. They can be emulated using a statement that is always true.

During parsing, switch statements get parsed as: Then, during bytecoding they get parsed as a collection of conditionals. During this stage, any equals signs that need inserted are inserted.

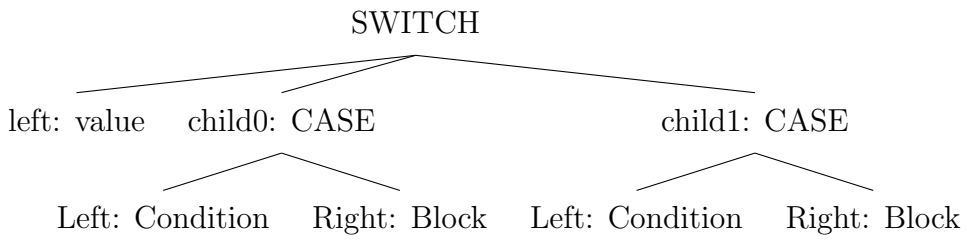


Figure 12: Parsed switch statement

```
1st Condition:
1. Load condition (not shown)
2. JUMP_IF_FALSE 5
3. 1st block (not shown)
4. JUMP 9
2nd Condition:
5. Load condition (not shown)
6. JUMP_IF_FALSE 9
7. 2nd block (not shown)
8. JUMP 9
9. After switch statement (not shown)
```

Figure 13: Bytecoded switch statement

7.6.2 Iteration Statements

An iterative statement is a statement that can run multiple times. In Aurum, there's two types of iterative statements: while loops, and for loops.

A while loop is a loop that will run as long as its condition is true. Below is a while loop in Aurum.

```
while (condition) {
 block;
}
```

The condition is checked before the block is ran. During parsing, while loops are parsed as:

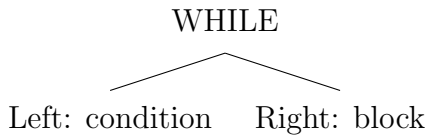


Figure 14: How while loops are parsed

During bytecoding, while loops are converted into the following format:

```
1. condition (not shown)
2. JUMP_IF_FALSE 5
3. block (not shown)
4. JUMP 1
5. after loop (not shown)
```

While loops are not guaranteed to run.

There's another type of iterative statement: for loops. They declare an initializer variable, and then a condition is checked every time the loop runs. After the loop, the variable is updated, and the condition checked again. Below is an example of a for loop in Aurum:

```
for (int i = 0; i < 20; i++) {
 block
}
```

Each for loop has three parts: `for (starting value; conditional; update)`.

During parsing, for loops are turned into:

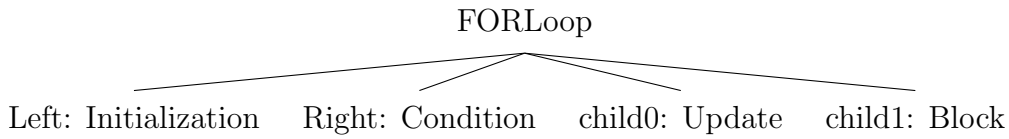


Figure 15: Parsed for loop

During bytecoding, for loops are converted to:

1. `Declares the iteration variable (not shown)`
2. `JUMP 6`
3. `Update variable (not shown)`
4. `Condition (not shown)`
5. `JUMP_IF_FALSE 8`
6. `block (not shown)`
7. `JUMP 3`
8. `After loop (not shown)`

For loops are guaranteed to run at least once.

There's two useful statements to keep in mind when making loops — break & continue. Break exits the loop immediately. Continue jumps back to the condition-check (In for loops, it jumps to the update block). In parsing, break & continue are parsed as seperate nodes. In bytecoding, they are converted into JUMP x.

7.6.3 Other statements

Below is a list of some other useful statements:

- `return x`; In a function, stops execution, and sends the value back to whatever called the function. In parsing, gets parsed as:

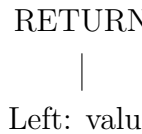


Figure 16: Return statement parsed

  In bytecoding, return gets its own opcode.

- `import "file.aur"`; used to import a file. Function declarations & struct declarations become available to this file. Has to have quotes surrounding the file name.

- `import @library`; used to import libraries. Has to have an @ before the library name.

8 Standard Library

The Aurum standard library has a total of 250 functions to facilitate program development. They are split into 7 libraries:

- Default - No import statement needed; these are global functions that developers can access anywhere. Has 41 functions.
- Math - To import, use `import @math;`. The Math library includes functions on base conversions, exponents/logarithms, geometry, trigonometry, number properties, and statistics. Has 83 functions. Has two sub-libraries that requires math to be imported:
  - Complex Numbers - To import, use `import @cplx`. The complex library includes the cplxNum struct, which supports operations with complex numbers, and various functions using complex numbers. Has 42 functions.
  - Linear Algebra - To import, use `import @lalg;`. This is the only library fully written in Aurum. Has structs for vector2's, vector3's, arbitrary-dimension vectors, and matrices. Has 52 functions.
- Time - To import, use `import @time;`. The time library includes functions on getting the current time, UTC formatting, and time. Has 4 functions.
- Io - To import, use `import @io;`. The io library includes functions on manipulating files and directories. Has 16 functions.
- Rand - To import, use`import @rand;`. The random library includes functions on generating random numbers, permutations, combinations, factorials (and $\Gamma()$). Has 10 functions.

8.1 Default Functions

These are functions that don't require any imports to use.

8.1.1 Console Functions

These functions allow a program to interface with the console.

- `printInt(string, int?) -> void` - Prints an integer to the console preceded by a string. Arg1 can be any integer type. Examples:

```
printInt("Number:", 5i) - Prints "Number: 5" to the console.
printInt("Test:", 19l) - Prints "Test: 19" to the console.
```

- `printDec(string, double?, int?) -> void` - Prints a decimal (up to arg2 places) to the console preceded by a string. Arg1 can be any decimal type (float, double, double128). Arg2 can be any integer type. Examples:

```
printDec("Number:", 5.1899i, 2); - Prints "Number: 5.19"
to the console.
printDec("Test:", 19.55ld, 9); - Prints "Test: 19.550000000"
to the console.
```

– `printString(string) -> void` - Prints a string to the console. Examples:

  `printString("\n"); - Prints newline to the console.`
  `printString("Hello World!") - Prints "Hello World!" to the console.`

– `print(string, any, any, any ...) -> void` - Prints a formatted string to the console. Formats are declared by using the % character, and then the type of the format. The format string can have an infinite amount of characters, and an infinite amount of specifiers. Below is a list of acceptable formats for print():

  – `%s` - String e.x `print("Message: %s", "Hello World!")` → "Message: Hello World!"
  – `%c` - Character e.x `print("Char: %c", 65c)` → "Char: A"
  – `%h` - Short e.x `print("Short: %h", 19s)` → "Short: 19"
  – `%i` - Int e.x `print("Int: %h", 190i)` → "Int: 190"
  – `%l` - Long e.x `print("long: %l", 1900l)` → "Long: 1900"
  – `%f` - Float e.x `print("Float: %h", 19.191f)` → "Float: 19.191"
  – `%d` - Double e.x `print("Double: %d", 19.191d)` → "Double: 19.191d"
  – `%D` - Double128 e.x `print("Double128: %h", 19ld)` → "Double128: 19"
  – `%a` - Array (Prints it in the format of `[a, b, c, ...]`)
  – `%o` - Struct (Prints it in the format of `{attr1 : value1, attr2 : value2 ...}`)
  – `%O_` - Signals to print function to print it in base 8 (octal). (Int specifiers only)
  – `%u_` - Signals to print function to print as an unsigned number. (Int specifiers only)
  – `%x_` - Signals to print function to print it in base 16 (hex). (Int specifiers only)
  – `%g_` - Signals to print function to print it as compact as possible (Using either normal notation or e-notation) (Float specifiers only)

– `scani(string) -> long` - Prints the string, and waits for the user to enter a integer. If the user enters an invalid character, returns 0. Example:

`scani("Enter a number: ") - Prints "Enter a number: " and pauses execution until the user enters a number.`

– `scand(string) -> double128` - Prints the string, and waits for the user to enter a long double. If the user enters an invalid character, returns 0. Example:

`scani("Enter a number: ") - Prints "Enter a number: " and pauses execution until the user enters a number.`

– `scan(string, . . .) -> bool` - Doesn't print anything. For each print specifier in the string, prompt the user. If the value entered is valid, then set the nth argument passed into scan to the value they entered. Invalid for the `%a` and `%o` specifiers. Return value is if the inputs entered is valid. Examples:

```
double a;
int b;
```
`scan("%d %i", a, b) - Prompts the user twice, 1st for a double, and 2nd for an integer. Sets the value of a to the first input, and the value of b to the 2nd.`

8.1.2 Type Conversions

These are functions that are used to convert between types.

- `strToInt(string) -> long` - Converts a string into a long. Example:

```
string n = "299.34";
strToInt(n) - Returns 299.
```

- `strToDec(string) -> double128` - Converts a string into a double128. Example:

```
string n = "299.34";
strToDec(n) - Returns 299.34.
```

- `scaleInt(int?, int?) -> int?` - Converts an integer of one size to another. 1st input is the integer, 2nd input is the size to convert it to. Acceptable values for the 2nd input:
  - 1 - Bool
  - 8 - Char
  - -8 - uChar
  - 16 - Short
  - -16 - uShort
  - 32 - Int
  - -32 - uInt
  - 64 - Long
  - -64 - uLong

  Example:

```
char n = 29c;
int m = scaleInt(n, 32); - m is now a integer with the value
of 29.
```

– `scaleDec(double?, int?) -> double?` - Converts a decimal of one size to another. 1st input is the decimal, 2nd input is the size to convert it to. Acceptable values for the 2nd input:

  – 32 - Float
  – 64 - Double
  – 128 - Double128

  Example:

```
float n = 29f;
double m = scaleDec(n, 64); - m is now a double with the value of 29.
```

– `newStr(any) -> string` - Converts a value to a string. Example:

```
float n = 29.555f;
string m = newStr(n) - m is now a string whose value is "29.555"
```

– `typeof(auto, optional bool) -> string` - Returns the type of an input. If the input is a number and arg1 is true, it will format number inputs as "int8", "uint64", "double32" instead of "char", "ulong", and "float". Structs passed in return their name. Arrays passed in returns the type of their first element and "[]". e.x `typeof([2,4,6,8]) -> "int[]"`.

8.1.3 Floats

These are functions that are used to manipulate floats.

- `isInf(float?), isNaN(float?), isNormal(float?) -> bool` - Returns if a float is infinity, NaN, or normal. Examples:

- `toExp(float?, float?), toSciNo(double?) -> floatConversion` - Converts a float into some form of $n * base^p$, where n, base, & p are numbers. Base is the 2nd parameter passed (10 in the case of toSciNo). floatConversion is a struct used to store this, containing a string representation of it, and a double[] representation. Examples:

```
Definition of the floatConversion struct:
struct floatConversion {
 double[] arrayConversion;
 string stringConversion;
 function floatConversion(double[] arr, string str) -> floatConversion
{
  this.arrayConversion = arr;
  this.stringConversion = str;
  return this;
 }
}


Examples:
float number = 2**19+59;
toExp(number, 2) - Converts it to some form of n*2^p
toSciNo(number) - Converts it to n*10^p.
```

8.1.4 Arrays

These are functions use to manipulate arrays.

- `array_push(any), array_append(any), array_insertElement(any, int?) -> void` - Inserts an element into an array. The index at which it inserts depends on the function called;

Function called	Where it inserts
array_push	Index 0
array_append	Last index
array_insertElement	At a index provided by arg1

  Examples:

```
int[] arr = [4,6,8,10];
arr.push(2);
arr.append(12);
print("%a", arr) - Prints "[2,4,6,8,10,12]"
```

- `array_pop(bool), array_removeElement(bool, int?) -> ?` - Removes an element from an array. If the boolean passed is true, then returns that element. If not, doesn't return it. Array_pop removes the first element, while array_removeElement removes at the index.

  Examples:

```
int[] arr = [3,4,5,6,7,8,9,10];
print("Removed %i", arr.pop(true));
arr.removeElement(false, 1);
print("Removed %i", arr.removeElement(true, 2));
arr.removeElement(false, 3);
print("%a", arr) - Prints "[4,6,8,10]"
```

- `array_reverse() -> void` - Reverses an array. Modifies the original array. Example:

```
int[] arr = [4,3,5,4,7,1,9,10];
arr.reverse();
print("\n%a\n", arr); ‘ prints [10,9,1,7,4,5,3,4]
```

– `array_sort(fnptr) -> array` - Sorts an array. The function passed through is a comparison function. The function passed through has to be declared as `function cmp(any a, any b) -> int`. If $cmp(a, b) > 0$, then that means b needs to be before a. If $cmp(a, b) < 0$, then that means a needs to be before b. Doesn't modify the original array.

```
function cmpstr(string a, string b) -> int {
 if (#a > #b) {return -1;}
 if (#a < #b) {return 1;}
 return 0;
}
function cmpstr2(string a, string b) -> int {
 if (#a > #b) {return 1;}
 if (#a < #b) {return -1;}
 return 0;
}
string[] arr = ["aaa", "b", "cc", "dddd"];
print("%a", arr.sort(&cmpstr)); ‘ ["dddd", "aaa", "cc", "b"]
print("%a", arr.sort(&cmpstr2)); ‘ ["b", "cc", "aaa", "dddd"]
```

8.1.5 Strings

These are functions used to manipulate strings. Additionally, strings have two operator-overloads, which are:

– +; concatenation. e.x `"Hello, " + "World!" = "Hello World!"`.

– ==; equality. e.x `"Hello" == "Hello"`.

Functions used to manipulate strings are:

– `string_toUpper(int), string_toLower(int), string_swapCase(int) -> string` - Manipulates the casing of the 1st arg0 characters. If arg0 is zero, applies the function to the whole string. Examples:

```
string n = "Hello World!";
print("\n%s\n", n.toUpper(3)) ‘ prints "HELlo World!".
print("\n%s\n", n.toLower(3)) ‘ prints "hello World!".
print("\n%s\n", n.swapCase(3)) ‘ prints "hELlo World!".
```

– `string_findOccurances(string) -> int[]` - Returns an array containing the indices that the string shows in up. If the string entered has multiple characters, the index returned is the index of the 1st character. Example:

```
string n = "The quick brown fox jumps over the lazy dog";
int[] o = n.toLower(0).findOccurances("the"); ‘ [0,31]
int[] p = n.toLower(0).findOccurances("e"); ‘ [2, 28, 33]
```

– `string_split(char) & string_splitStr(string) -> string[]` - Splits a string by a character (Or in the case of `string_splitStr`, a string). Any characters after the last instance of the filter is also added to the string. Example:

```
string n = "The quick brown fox jumps over the lazy dog";
string[] a = n.split(" "); ‘ ["The", "quick", "brown", "fox",
"jumps", "over", "the", "lazy", "dog"]
string[] b = n.split("e "); ‘ ["Th", "quick brown fox jumps
over th", "lazy dog"]
```

– `string_eqSplit(int n) & string_nSplit(int n) -> string[]` - eqSplit splits the string into equal parts whose length is n. nSplit splits the string into n equal parts. Any part of the string that doesn't follow that function's rule is also added. Example:

```
string n = "The quick brown fox jumps over the lazy dog";
string[] a = n.eqSplit(5); ‘ ["The quic", "k brown ", "fox
jump", "s over t", "he lazy ", "dog"]; 5 equal parts, and leftover
characters.
string[] b = n.nSplit(5); ‘ ["The q", "uick ", "brown", "
fox ", "jumps", " over", " the " "lazy " "dog"]; Parts whose
length is 5, and leftover characters.
```

– `string_leftPad(int, char), string_rightPad(int, char) -> string` - Adds a character arg0 times to either the start (leftPad), or end (rightPad), of a string. Example:

```
string n = "Hello World!";
string m = n.leftPad(3, 'a'); ‘ "aaaHello World!"
string o = n.rightPad(3, 'a'); ‘ "Hello World!aaa"
```

- `string_replace(string, string) -> string` - Replaces all instances of arg0 with arg1 in the string. Example:

```
string n = "The quick brown fox jumps over the lazy dog";
string m = n.toLower(0).replace("the", "string"); ‘ "string quick brown fox jumps over string dog"
```

- `formats(string, . . .) -> string` - Works exactly like print(), but doesn't print it to the console and instead returns it as a string. Example:

```
int a = 2;
int b = 5;
int c = 10;
string quadratic = formats("(-%i\pmsqrt(%i^2-4*%i*%i)/2*%i)", b, b, a, c, a);
print("\n%s", quadratic) ‘ prints "(-5±sqrt(5^2-4*2*10)/2*2)"
```

8.1.6 Error Handling

The Aurum standard library includes some tools for error-handling. It doesn't have try-catch statements yet, but that will come in a future update.

- `exception(int, string, int) -> exception` - Creates a new exception
- `throw() -> void` - Prints the error and exits the program.
- `exit() -> void` - Exits the program.

8.2 @math Library

These are mathematical functions and constants.

8.2.1 Math Constants

The following are the constants included with @math. They will not include usage examples.

- `const float M_PI8;` - Mathematical constant of π to 8 digits.
- `const double M_PI16;` - Mathematical constant of π to 16 digits.
- `const double128 M_PI32;` - Mathematical constant of π to 32 digits.
- `const double M_PI;` - Shorthand for `M_PI16`.

- `const float M_E8;` - Mathematical constant of e to 8 digits.
- `const double M_E16;` - Mathematical constant of e to 16 digits.
- `const double128 M_E32;` - Mathematical constant of e to 32 digits.
- `const double M_E;` - Shorthand for `M_E16`.

- $\tau = 2\pi \approx 6.28318$
- `const float M_TAU8;` - Mathematical constant of τ to 8 digits.
- `const double M_TAU16;` - Mathematical constant of τ to 16 digits.
- `const double128 M_TAU32;` - Mathematical constant of τ to 32 digits.
- `const double M_TAU;` - Shorthand for `M_TAU16`.

- `const float M_SQRT28;` - Mathematical constant of $\sqrt{2}$ to 8 digits.
- `const double M_SQRT216;` - Mathematical constant of $\sqrt{2}$ to 16 digits.
- `const double128 M_SQRT232;` - Mathematical constant of $\sqrt{2}$ to 32 digits.

– `const double M_SQRT2;` - Shorthand for `M_SQRT216`.

– $\phi = GoldenRatio = \frac{1+\sqrt{5}}{2} \approx 1.618033$

– `const float M_GRAT8;` - Mathematical constant of ϕ to 8 digits.

– `const double M_GRAT16;` - Mathematical constant of ϕ to 16 digits.

– `const double128 M_GRAT32;` - Mathematical constant of ϕ to 32 digits.

– `const double M_GRAT;` - Shorthand for `M_GRAT16`.

– `const float M_SSUBNORMALF;` - Smallest subnormal float ($\approx 2^{-149}$).

– `const double M_SSUBNORMALD;` - Smallest subnormal double ($\approx 2^{-1074}$).

– `const double128 M_SSUBNORMALLD;` - Smallest subnormal double128 ($\approx 2^{-16495}$).

– `const float M_LSUBNORMALF;` - Largest subnormal float ($\approx (2-(2^{-23}))*2^{-126}$).

– `const double M_LSUBNORMALD;` - Largest subnormal double ($\approx (2-(2^{-52}))*2^{-1022}$).

– `const double128 M_LSUBNORMALLD;` - Largest subnormal double128 ($\approx (2-(2^{-112}))*2^{-16382}$).

– `const float M_EPSILONF;` - Gap between 1 & the next larger float ($\approx 2^{-23}$).

– `const double M_EPSILOND;` - Gap between 1 & the next larger double ($\approx 2^{-52}$).

– `const double128 M_EPSILONLD;` - Gap between 1 & the next larger double128 ($\approx 2^{-112}$).

- `const float M_FMAX;` - Largest non-infinity float ($\approx (2-(2^{-23}))*2^{127}$).
- `const double M_DMAX;` - Largest non-infinity double ($\approx (2-(2^{-52}))*2^{1023}$).
- `const double128 M_LDMAX;` - Largest non-infinity double128 ($\approx (2-(2^{-112}))*2^{16383}$).
- `const float M_FMIN;` - Smallest non-infinity float ($-M\_FMAX$).
- `const double M_DMIN;` - Smallest non-infinity double ($-M\_DMAX$).
- `const double128 M_LDMIN;` - Smallest non-infinity double128 ($-M\_LDMAX$).
- `const char M_MAX8 = const char M_MAXC = 127` - Maximum value that the char datatype can store.
- `const char M_MAX8U = const char M_MAXUC = 127` - Maximum value that the uchar datatype can store.
- `const short M_MAX16 = const short M_MAXS = 16383` - Maximum value that the short datatype can store.
- `const ushort M_MAX16U = const ushort M_MAXUS = 32767` - Maximum value that the ushort datatype can store.
- `const int M_MAX32 = const int M_MAXI = 0x7fffffff` $\approx 2.1*10^9 = 2^{31}-1$ - Maximum value that the int datatype can store.
- `const uint M_MAX32U = const uint M_MAXUI = 0xffffffff` $\approx 4.2*10^18 = 2^{32}-1$ - Maximum value that the uint datatype can store.
- `const long M_MAX64 = const long M_MAXL = 0x7fffffffffffffff` $\approx 9.2*10^18 = 2^{63}-1$ - Maximum value that the long datatype can store.
- `const uint M_MAX32U = const uint M_MAXUI = 0xffffffffffffffff` $\approx 1.8*10^19 = 2^{64}-1$ - Maximum value that the ulong datatype can store.

- Minimum integer size constants also exist; they're in the foramt of M_MIN{Size} or M_MIN{Initial} (e.x M_MINC is -127, and M_MIN16 is -16383.)

8.2.2 Base Conversions

These are functions for manipulating bases. These DO work with decimal numbers, as well as integers. For these bases, it uses the following character set: `0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz+/\\!@#$%^&*_-=;` along with some other characters (see mathFunctions.c). Supports bases from 2-128.

- `decToBase(double?, int, optional string) -> string` - Converts a decimal number into base n. If the string is entered, uses a custom set of digits. also Examples:

```
decToBase(2**9+45, 19); ‘ Returns "1A6"
decToBase(165,2); ‘ Returns "10100101";
decToBase(M_PI32, 3); ‘ Returns "10.01021101222201021100"
```

- `baseToDec(string, int) -> double128` - Converts a base n string back into base 10. When a 2nd string is entered, uses a custom set of digits. Examples:

```
baseToDec("1A6", 19ld); ‘ Returns 557
baseToDec("10100101", 2ld); ‘ Returns 165
baseToDec("10.01021101222201021100", 3ld); ‘ Returns 3.141593.
```

- Along with decToBase() & baseToDec(), there are several more functions that make decToBase & baseToDec more readable:
  - `binToDec(string) -> double128 & decToBin(double128) -> string` - Functions for converting to and from binary (base 2). Uses the digits `01`.

- `octToDec(string) -> double128 & decToOct(double128) -> string` - Functions for converting to and from octal (base 8). Uses the digits `01234567`.
- `dozToDec(string) -> double128 & decToDoz(double128) -> string` - Functions for converting to and from dozenal (base 12). Uses the digits `0123456789AB`.
- `hexToDec(string) -> double128 & decToHex(double128) -> string` - Functions for converting to and from hexadecimal (base 16). Uses the digits `0123456789ABCDEF`.
- `B64ToDec(string) -> double128 & decToB64(double128) -> string` - Functions for converting to and from base 64. Uses the base 64 digits `(0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz+/)`

8.2.3 Exponents

There are several functions in @math that can be used to manipulate logarithms.

- `log(double?, double?) -> double?` - Computes $\log_{arg1}(arg0)$.
- `log2(double?) -> double?` - Computes $\log_2(arg0)$.
- `log10(double?) -> double?` - Computes $\log_{10}(arg0)$.
- `ln(double?) -> double?` - Computes $\ln(arg0)$ ($\log_e(arg0)$).

Logarithm Examples:

```
log(128, 4) - Returns 3.5. 4^3.5 = 128.
log2(256.25) - Returns 8.001.
log10(100) - Returns 2
ln(M_E**5); - Returns 5
```

8.2.4 Geometry & Trigonometry

There are several functions in @math that are used to do geometry.

- There's two functions for radian-degrees conversion: `dToR(double?) & rToD(double?) -> double?`. dToR converts from degrees to radians, and rToD converts from radians to degrees. Examples:

```
dToR(45); - 0.785398
rToD(0.785398) - 45
```

- There's several functions in Aurum for dealing with trignometric functions. Due to the amount of these, they will only have short descriptions & no usage examples. Functions ending in "d" take their argument in as degrees, not radians. Functions with an "h" at the end are hyperbolic functions. Functions starting with an "a" are inverse functions.
  - `sin(double?), sind(double?) -> double?` - Computes $\sin\theta$.
  - `sinh(double?), sinhd(double?) -> double?` - Computes $\sinh\theta$.
  - `asin(double?), asind(double?) -> double?` - Computes $\arcsin\theta$. Throws an error for values outside of the range $[1, -1]$.
  - `asinh(double?), asinhd(double?) -> double?` - Computes $\sinh^{-1}\theta$.
  - `cos(double?), cosd(double?) -> double?` - Computes $\cos\theta$.
  - `cosh(double?), coshd(double?) -> double?` - Computes $\cosh\theta$.
  - `acos(double?), acosd(double?) -> double?` - Computes $\arccos\theta$. Throws an error for values outsode of the range $[1, -1]$
  - `acosh(double?), acoshd(double?) -> double?` - Computes $\cosh^{-1}\theta$. Throws an error for values outside the range $[1, \infty)$.
  - `tan(double?), tand(double?) -> double?` - Computes $\tan\theta$. Throws an error for od multiples of $\frac{\pi}{2}$
  - `tanh(double?), tanhd(double?) -> double?` - Computes $\tanh\theta$.

- `atan(double?), atand(double?) -> double?` - Computes $\arctan\theta$.
- `atanh(double?), atanhd(double?) -> double?` - Computes $\tanh^{-1}\theta$.

- `cot(double?), cotd(double?) -> double?` - Computes $\cot\theta$. Throws an error for odd multiples of $\frac{\pi}{2}$. (90 for cotd)
- `coth(double?), cothd(double?) -> double?` - Computes $\coth\theta$. Throws an error for 0.
- `acot(double?), acotd(double?) -> double?` - Computes $\cot^{-1}\theta$.
- `acoth(double?), acothd(double?) -> double?` - Computes $\coth^{-1}\theta$. Throws an error for values outside the range $[-\infty,-1]\cup[1,\infty]$

- `sec(double?), secd(double?) -> double?` - Computes $\sec\theta$. Throws an error for odd multiples of $\frac{\pi}{2}$ (90 for secd)
- `sech(double?), sechd(double?) -> double?` - Computes $sech\theta$.
- `asec(double?), asecd(double?) -> double?` - Computes $\sec^{-1}\theta$. Throws an error for values outside of the range $(-\infty,-1]\cup[1,\infty)$
- `asech(double?), asechd(double?) -> double?` - Computes $sech^{-1}\theta$. Throws an error for values outside of the range $[-1,1]$, or if the value is 0.

- `csc(double?), cscd(double?) -> double?` - Computes $\csc\theta$. Throws an error for odd multiples of π (180 for cscd).
- `csch(double?), cschd(double?) -> double?` - Computes $csch\theta$. Throws an error for 0.
- `acsc(double?), acscd(double?) -> double?` - Computes $\csc^{-1}\theta$. Throws an error for values outside of the range $(-\infty,-1]\cup[1,\text{inf})$
- `acsch(double?), acschd(double?) -> double?` - Computes $csch^{-1}\theta$. Throws an error for zero.

– Along with trigometric ratios, Aurum also has two functions that implement the Pythagorean Theorem ($a^2 + b^2 = c^2$).

  – `hypot(double?, double?) -> double?` - Computes the hypotenuse of a triangle using two side lengths. `c = \sqrt{a^2 + b^2}`. Example:

    ```
    hypot(3,4) - Returns 5.
    ```

  – `sidel(double? double?) -> double?` - When a triangle's side-length & hypotenuse are known, sidel() returns the length of the other side. arg0 is the side length, arg1 is the hypotenuse. Example:

    ```
    sidel(4,5) - Returns 3.
    sidel(3,5) - Returns 4.
    ```

8.2.5 Numbers

There's several functions that relate to a number's properties.

– `gcd(int?, int?) -> int?` - Computes the greatest common factor of two numbers:

– `lcm(int?, int?) -> int?` - Computes the least common multiple of two numbers.

```
gcd(54,24) - Returns 6.
lcm(54,24) - Returns 216.
```

– `isPrime(int?) -> bool` - Returns if a number is prime or not. Example:

```
for (int i = 0; i < 20; i++) {
 if (isPrime(i)) {
  print("%i\n", i); - Prints "1 2 3 5 7 11 13 17 19".
 }
}
```

– `primeFac(int?) -> int?[]` - Returns the prime factors of a number. If the number is prime, 1 isn't included. Any exponents in the prime factorization are split. Example:

```
primeFac(5896) - Returns [2,2,2,11,67].
```

8.2.6 Statistics & Array Operations

There are several functions that help with statistics in Aurum.

– `array_sum() & array_prod() -> double?` - Computes the sum (or product) of an number array. If a non-number element is found, does throw an error. Example:

```
int[] arr = [2,4,6,8,10];
arr.prod(); ‘ 3840
arr.sum(); ‘ 30
```

– `array_mean() -> double?` - Computes the mean (average) of an array. If a non-number element is found, throws an error.

```
int[] arr = [2,4,6,8,10];
arr.mean() ‘ 15
```

– `array_median() -> double?` - Computes the median of an array. If a non-number element is found, throws an error.

```
int[] arr = [2,4,6,8,10];
arr.median() ‘ 7
```

– `array_quantiles(int n) -> double?[]` - Computes the nQuantile for an array of numbers. If a non-number element is found, throws an error. Example:

```
‘ Computes the deciles of a random 20-item set:
long[] arr = [0];
srand(time());
for (int i = 0; i < 20; i++) {
 arr.append(randL(0,100));
}
print("\n%a", arr);
print("\n%a, arr.quantiles(10));
```

8.3 @cplx Library

Along with having functions for the Real numbers, Aurum also has several functions for complex numbers. To use these functions, having @math imported is required.

8.3.1 cplxNum Struct

To declare a new complex number, use the cplxNum struct, listed below:

```
struct cplxNum {
 double128 a;
 double128 b;
 function cplxNum(double128 a, double128 b) -> cplxNum {
  this.a = a;
```

```
    this.b = b;
    return this;
   }
   ‘ Several functions have been cut out from this definition (see
below).
  }
  ";
```

Any function in this library that returns a real number will return a double128, as that's what cplxNum uses. Below are functions called by using the cplxNum struct.

- `cplxNum_cabs() -> double128` - Returns a complex number's distance from 0.
- `cplxNum_op_len() -> double128` - Operator overload for `#`. Same as calling cplxNum.cabs().

```
cplxNum a = cplxNum(19ld, 23ld);
a.cabs() ‘ 28.31
#a ‘ 28.31
```

- `cplxNum_carg() -> double128` - Returns a complex number's argument. (Angle between positive real axis & x.)

```
cplxNum a = cplxNum(-3, -1.1);
a.carg(); ‘ 0.35 radians
```

- `cplxNum_imag(), cplxNum_real() -> double128` - Return a complex number's real or imaginary parts.

```
cplxNum a = cplxNum(19ld, 23ld);
a.real(); ‘ 19
a.a ‘ 19; a.real() = a.a
a.imag(); ‘ 23
a.b; ‘ 23; a.imag() = a.b
```

– `cplxNum_toPolarString() -> string` - Converts a complex number to the form $re^{i\theta}$.

– `cplxNum_toCartesianString() -> string` - Converts a complex number to the form $a + bi$.

```
cplxNum a = cplxNum(19ld, 23ld);
a.toPolarString(); ‘ 29.83^i0.88
a.toCartesianString(); ‘ 19+23i
```

– `cplxNum_conj() -> cplxNum` - Takes a complex number's conjugate (negates the imaginary part).

– `cplxNum_op_add(cplxNum) -> cplxNum` - Addition overload operator. Adds two complex numbers.

– `cplxNum_op_sub(cplxNum) -> cplxNum` - Subtraction overload operator. Subtracts two complex numbers.

– `cplxNum_op_mul(cplxNum) -> cplxNum` - Multiplication overload operator. Multiplies two complex numbers.

– `cplxNum_op_exp(cplxNum) -> cplxNum` - Exponentiation overload operator. Exponentiates two complex numbers. Same thing as calling cpow().

```
cplxNum a = cplxNum(2,4);
cplxNum b = cplxNum(3,5);
a.op_add(b); ‘ 5+9i
a.op_sub(b); ‘ -1-i
a.op_mul(b); ‘ -14+22i
```

8.3.2 Complex Exponents

In @cplx, there are several functions for dealing with complex exponents & logarithms.

- `cpow(cplxNum, cplxNum) -> cplxNum` - Returns $arg0^{arg1}$.
- `cexp(cplxNum) -> cplxNum` - Returns e^{arg0}.
- `ctxp(cplxNum) -> cplxNum` - Returns 2^{arg0}.
- `clog(cplxNum, cplxNum) -> cplxNum` - Returns $\log_{arg1}(arg0)$.
- `cln(cplxNum) -> cplxNum` - Returns $\ln arg0$ ($\log_e(arg0)$).
- `clog2(cplxNum) -> cplxNum` - Returns $\log_2(arg0)$.
- `clog10(cplxNum) -> cplxNum` - Returns $\log_{10}(arg0)$.

```
cplxNum a = cplxNum(10ld, 30ld);
cplxNum b = cplxNum(0, 5ld);
cpow(a, b) ‘ ~0 - 0.00193i
clog(a, b) ‘ 1.487-0.675224i
```

8.3.3 Complex Trigonometry

These are essentially just the trigonometric functions from @math expanded to the $\mathbb{C}$ domain.

- `csin(cplxNum) -> cplxNum` - Computes complex $\sin z$
- `ccos(cplxNum) -> cplxNum` - Computes complex $\cos z$
- `ctan(cplxNum) -> cplxNum` - Computes complex $\tan z$
- `csec(cplxNum) -> cplxNum` - Computes complex $\sec z$
- `ccot(cplxNum) -> cplxNum` - Computes complex $\cot z$
- `ccsc(cplxNum) -> cplxNum` - Computes complex $\csc z$
- `casin(cplxNum) -> cplxNum` - Computes complex $\arcsin z$
- `cacos(cplxNum) -> cplxNum` - Computes complex $\arccos z$
- `catan(cplxNum) -> cplxNum` - Computes complex $\arctan z$

– `casec(cplxNum) -> cplxNum` - Computes complex $\sec^{-1}(z)$

– `cacot(cplxNum) -> cplxNum` - Computes complex $\cot^{-1}(z)$

– `cacsc(cplxNum) -> cplxNum` - Computes complex $\csc^{-1}(z)$

- `csin(cplxNum) -> cplxNum` - Computes complex $\sinh z$
- `ccos(cplxNum) -> cplxNum` - Computes complex $\cosh z$
- `ctan(cplxNum) -> cplxNum` - Computes complex $\tanh z$
- `csec(cplxNum) -> cplxNum` - Computes complex $sech(z)$
- `ccot(cplxNum) -> cplxNum` - Computes complex $coth(z)$
- `ccsc(cplxNum) -> cplxNum` - Computes complex $cschz$
- `casin(cplxNum) -> cplxNum` - Computes complex $\sinh^{-1} z$
- `cacos(cplxNum) -> cplxNum` - Computes complex $\cosh^{-1} z$
- `catan(cplxNum) -> cplxNum` - Computes complex $\tan^{-1} z$
- `casec(cplxNum) -> cplxNum` - Computes complex $sech^{-1}(z)$
- `cacot(cplxNum) -> cplxNum` - Computes complex $coth^{-1}(z)$
- `cacsc(cplxNum) -> cplxNum` - Computes complex $csch^{-1}(z)$

8.4 @lalg Library

This library has several functions & structs for dealing with vectors & matrices. This, like @cplx, requires @math to be imported to use.

8.4.1 Vectors

Below are the vector2, vector3, & vectorN structs.

```
struct vector2 {
 double128 x;
 double128 y;
 function vector2(double128 x, double128 y) -> vector2 {
  this.x = x;
  this.y = y;
  return this;
 }
}
```

```
struct vector3 {
 double128 x;
 double128 y;
 double128 z;
 function vector3(double128 x, double128 y, double128 z) -> vector3
{
  this.x = x;
  this.y = y;
  this.z = z;
  return this;
}
struct vectorN {
 double128[] values;
 function vectorN(double128[] values) -> vectorN {
  this.values = values;
  return this;
 }
}
```

For vectors, there are also three additional constants used for referring to axes. Most of these functions won't have examples associated with them, as they are quite simple.

```
const int xAxis = 1;
const int yAxis = 2;
const int zAxis = 3;
```

Here are the functions for dealing with vectors.

- `vector?_magnitude()` - Returns the magnitude of a vector.
- `vector?_dotProduct(vector?)` - Computes the dot product of a vector.
- `vector2_crossProduct(vector2) -> vector2, vector3_crossProduct(vector3) -> vector3` - Computes the cross product of a vector.
- `vector?_angle(vector?) -> double128` - Returns the angle between two vectors.
- `vector?_round(int) -> double128` - Returns a copy of the vector, rounded to n digits.

- `vector?_normalize() -> vector?` - Normalizes a vector (Makes the vector's magnitude 1).
- `vector?_normalizeN(double128) -> vector?` - Makes a vector's magnitude n.
- `vector2_rotate(double128) -> vector2` - Rotates a vector by θ counterclockwise around the origin.
- `vector3_rotate(int, double128) -> vector3` - Rotates a vector around an axis. 1st argument has to be one of the axis constants.
- `vector3_rotateByVector3(int[], vector3) -> vector3` - Rotates a vector in the following order: arg0_0, arg0_1, arg0_2. All elements of arg0 have to be axes. The following code rotates a vector π radians around the y axis, π radians around the z axis, then $\frac{\pi}{2}$ radians around the x axis:

```
vector3 n = vector3(5ld, 6ld, 7ld);
n.rotateByVector3(vector3(M_PI32/2ld, M_PI32, M_PI32), [yAxis,
zAxis, xAxis]).toString();
```

- `vector?_toString() -> string` - Converts a vector to a string in the following format: ¡x, y, z, w, ... ¿.
- `vector?_op_add(vector?) -> vector?` - Addition operator overload; Adds two vectors.
- `vector?_op_sub(vector?) -> vector?` - Subtraction operator overload; Subtracts two vectors.
- `vector?_op_neg(vector?) -> vector?` - Negation operator overload; Rotates the vector by 180 degrees on all axes.
- `vector?_scalarMult(double128) -> vector?` - Multiplies all elements of the vector by n.
- `vector?_op_len -> double128` - `#` Operator Overload. Returns the magnitude of a vector.
- `vectorN_dims() -> int` - Returns the amount of dimensions in a vectorN.

8.4.2 Matrices

In addition to being able to deal with vectors, @lalg also has its own matrix struct:

```
struct mtrx {
 double128[] data; ‘ Multidimensional array
 int rows;
 int cols;
 function mtrx(double128[] data, int rows, int cols) -> mtrx {
  this.data = data;
  this.rows = rows;
  this.cols = cols;
  return this;
 }
}
```

Below is an example of making a 2 row by 3 column matrix, then accessing element at matrix index $newMatrix_{12}$:

```
mtrx newMatrix = mtrx([[1ld,2ld,3ld],[4ld,5ld,6ld]], 2, 3);
newMatrix[0][1]; ‘ Accesses matrix index 12.
```

Below is the functions that are used when dealing with matrices:

- `doubleMtrxFromDims(int, int) -> double128[]` - Makes a $arg0xarg1$ multidimensional double128[]. All entries are 0.
- `identityMtrx(int) -> mtrx` - Makes the identity matrix of dimension arg0. (The identity matrix is a matrix where the diagonal is 1, and the rest are zeroes.)
- `mtrx_det() -> double128` - Gets the determinant of a matrix. Throws an error if the matrix isn't square.
- `mtrx_toVector() -> vectorN` - Converts a $1xN$ or $Nx1$ matrix into a vectorN.
- `mtrx_toString() -> string` - Converts a matrix into a string in the following format:

```
[[11, 12, 13],
 [14, 15, 16]]
```

- `mtrx_mult(mtrx) -> mtrx` - Multiplication operator overload; Multiplies two matrices. If they are unable to be multiplied, throws an error.
- `mtrx_round(int) -> mtrx` - Returns a copy of the 1st matrix rounded to n places.
- `mtrx_scalarMult(double128) -> mtrx` - Multiplies elements of the matrix by n.
- `mtrx_op_add(mtrx) -> mtrx` - Addition operator overload; Adds two matrices
- `mtrx_op_sub(mtrx) -> mtrx` - Subtraction operator overload; Subtracts two matrices.
- `mtrx_op_mul(auto) -> mtrx` - Multiplication operator overload; If the parameter passed in is a scalar value, calls mtrx.scalarMult. Otherwise, does mtrx_mult.

8.5 @time Library

These are functions for dealing with time.

- `time() -> long` - Returns the amount of seconds since January 1st, 1970 (The Unix Epoch).
- `timeld() -> double128` - Returns the amount of seconds since January 1st, 1970, including fractional parts of seconds.
- `clock() -> long` - Returns the clock cycle of the program (includes the entire interpretation process too)
- `sleep(int) -> void` - Stops execution of the program for arg0 milliseconds.
- `getUTC() & toUTC(long) -> UTCTime` - These both are functions for handling UTC time. getUTC() gets the current time, and converts it to a UTCTime struct. getUTC() does include the current milliseconds, but toUTC(long) does not. Both of them use the UTCTime struct, shown here:

```
struct UTCTime {
 int ms;
 int second;
 int minute;
 int hour;
 int mday;
 int month;
 int year;
 int wday;
 int yday;
 bool isdst;
};";
```

8.6 @rand Library

The @rand library provided functions for random number generations, permutations/combinations, & factorials.

8.6.1 Random Number Generation

- `srand(long) -> void` - Seeds the random number generator. This has to be called before any random number generation. A good way to ensure inconsistent random numbers are generated is by calling `srand(time(NULL))`, if you have @time imported.
- `rand() -> long` - Returns a random integer between 0 & 32767.
- `randL(long, long) -> long` - Generates a random long between the range $[minimum, maximum]$
- `randLD(double128, double128) -> double128` - Generates a random double128 between the range $[minimum, maximum]$.

8.6.2 Factorial & Gamma Functions

- `factorial(long) -> long` - Computes the basic factorial function. Domain is $[n|n \in \mathbb{N}]$

– `gamma(double128) -> double128` - Computes the gamma function. Essentially, it's the factorial function extended to all real numbers. Domain is $[n|n \in \mathbb{R}]$

– `lgamma(double128) -> double128` - Computes $\ln(|\Gamma(n)|)$.

8.6.3 Permutations, Combinations, and Array.Choose()

– `permutations(int, int, bool)` - Computes the amount of permutations in a set of size arg0, choosing arg1 items, and if repetition is allowed.

– `combinations(int, int, bool)` - Computes the amount of combinations in a set of size arg0, choosing arg1 items, and if repetition is allowed.

– `array_choose() -> any` - Returns `array[randL(0, #array)]`.

8.7 @io Library

@io includes several functions for manipulating files. Only works with windows systems.

8.7.1 Opening Files

In the @io library, the file struct is used often, so here is the definition for the file struct:

```
   const int FM_READ = 1i; ‘ Only allows reading of the file. Doesn’t
create the file if it’s not there.
   const int FM_WRITE = 2i; ‘ Overwrites the file, and only allows
writing. Creates the file if it’s not there.
   const int FM_APPEND = 3i; ‘ Doesn’t overwrite the file. Sets the
pos attribute of the file to the last character. Allows writing to
the end of the file.
   const int FM_READWRITE = 4i; ‘ Sets pos attribute to the start
of the file. Allows reading/writing.
   const int FM_READOVERWRITE = 5i; ‘ Overwrites the file, and allows
reading and writing. Sets the pos to zero. If the file doesn’t exist,
create it.
```

```
   const int FM_READAPPEND = 6i; ‘ Doesn’t overwrite the file, and
allows reading and appending. Sets the pos to the last character.
   const int EOF = -1i;
   struct file {
    long ptr;
    long pos;
    int mode;
    string name;
   }
```

- `fopen(string, int) -> file` - Opens a file using the arg0 as the path. Constructs a file struct with the data made. file.ptr is a pointer to the file struct inside of the VM. Do not change file.ptr unless you know what you’re doing. Arg1 is the mode that you are opening the file with.
- `file_close() -> void` - Closes the file. Always use this when you are done using a file. Sets file.ptr to 0.

8.7.2 Reading Files

- `file_getChar() -> int` - Gets the char at file.pos, and advance it 1. If the position reaches the end of the file, return EOF (-1).
- `file_getStr(int) -> string` - Reads arg0 characters and returns it as a string. Advances file.pos by that amount.
- `file_getLine() -> string` - Reads until it encounters a newline character.
- `file_read() -> string` - Reads the entire file. Doesn’t modify file.pos.
- `file_size() -> long` - Gets a file’s size in bytes.

8.7.3 Modifying Files

- `file_putChar(char) -> void` - Puts 1 character into the file, and advances pos by 1.

- `file_putStr(string) -> void` - Puts an entire string into the file, and advances pos by however long that string was.
- `file_writef(string, *) -> void` - Calls formats() with the arguments provided, and writes the string into it. Example:

```
    file n = fopen(path, "w");
    n.writef("%i+%i=%i", 1, 1, 2); ‘ This writes "1+1=2" to
the file.
    n.close();
```

- `file_rename(string) -> void` - Renames the file. This does change file.ptr.
- `file_delete() -> void` - Closes, then deletes, the file. Sets file.ptr to 0.

8.7.4 Directories

- `createDirectory(string) -> void` - Creates a directory.
- `deleteDirectory(string) -> void` - Deletes a directory.
- `renameDirectory(string, string) -> void` - Renames a directory from arg0 to arg1.
- `getAllFiles(string) -> directorySearch` - Grabs all file names & folder names from a directory, and returns them as a directorySearch.

```
    struct directorySearch {
     string[] files;
     string[] childDirs;
    }
```

9 AurX Executable Format

9.0.1 AurX Format Usage

AurX is a file format standard for executable Aurum files. Essentially, it enables bytecode to be saved in order to share programs easier, and to save time when repeatedly running programs. To use it, do `aurum file.aur -x`. To run an AurX file, do `aurum file.aurx`. After bytecoding, if -x is passed in, it will convert the bytecode into an AurX file with the same name as the file that ran. Other files that were imported will also be included in the AurX file. Each component of the bytecode is stored in different ways. Below is how function declarations are stored:

9.0.2 AurX Format Standard

Bytes:	Type	Data	Total
1	int8	Length of the function's name	1
1	int8	Function arguments	2
1	bool	If the function is user-made or system.	3
1	int8	Length of the return struct. 0 if it's a primitive type.	4
x	string	Function's return struct	4+x

Below is how global variables are stored:

Bytes:	Type	Data	Total
1	int8	Length of variable name	1
x	string	Variable name	1+x
1	int8	Variable type (Number, array, or struct, NOT the actual datatype)	x+2
1	bool	If the variable is an array	x+3
1	bool	If the variable is a constant	x+4
1	int8	Length of the variable's struct type. 0 if it's a primitive type.	x+5
y	string	Variable's struct type.	x+y+5

Below is how numeric constants are stored:

Bytes:	Type	Data	Total
1	int8	Constant type (Number, Array, or struct); Here it's a constant of 0, since it's a number.	1
32	Generic Number Struct	Stores the number value	33

Below is how array constants are stored:

Bytes:	Type	Data	Total
1	int8	Constant type (Number, array, or struct); here it's a constant of 1, since it's an array.	1
1	int8	Array type (Character array, number array, or other). This is used within the VM to distinguish between number arrays & strings.	2
4	int32	Amount of elements in the array	6
x	N/A	Member elements of the array. Stored in the same way as arrays & numbers are.	6+x

Below is how struct definitions are stored:

Bytes:	Type	Data	Total
1	int8	Length of the struct's name	1
x	string	Struct's name	1+x
1	int8	Amount of fields the struct has	2+x
1+y	string	Field names of the struct. y is the amount of characters, and the 1st byte of this is the amount of characters the name is.	3+x+y
(4 OR 5+z)*w	StructType	The type of a struct's field. w is the amount of fields in the struct. If the struct's field type is another struct, then it uses the 5+z size, otherwise it uses 4.	3+x+y+((4 OR 5+z)*w)

Below is how instruction-chunks are stored:

Bytes:	Type	Data	Total
4	int32	Amount of instructions in the chunk	1
1	int8	Length of the chunk's name	5
x	string	The chunk's name	5+x
(1+(0—2))*y	instruction[]	The list of instructions in the chunk. Each instruction has either 0, 1, or 2 arguments, and y is the total amount of instructions.	5+x+(1+(0—2)*y).

10 Planned additions to the Aurum Standard

- Speed optimizations — Despite the improvements, the language is still kind of slow. (Still much slower than python, even)
- Better operator overloading — Make things like (a+b).memberFunc() possible.
- 16, 32, 64 bit strings — These would be in a new @wstr library.
- Application library — @apl would allow the development of windows-based applications.
- Better localization — Right now, Aurum is only really well localized for Windows 1252 ASCII.